

TUGAS 4 LAB. BASIS DATA LANJUT

Nama: Faiz Faqih Haidar

NPM: 222310047

Kelas: TI-22-PB

1. Buat dan jalankan perintah untuk mencari data dari collection Customer menggunakan minimal 2 buah variasi dari perintah \$in dan \$nin.

The image displays two screenshots of the MongoDB Playground interface, showing various MongoDB commands and their results.

Top Screenshot:

- Commands:**

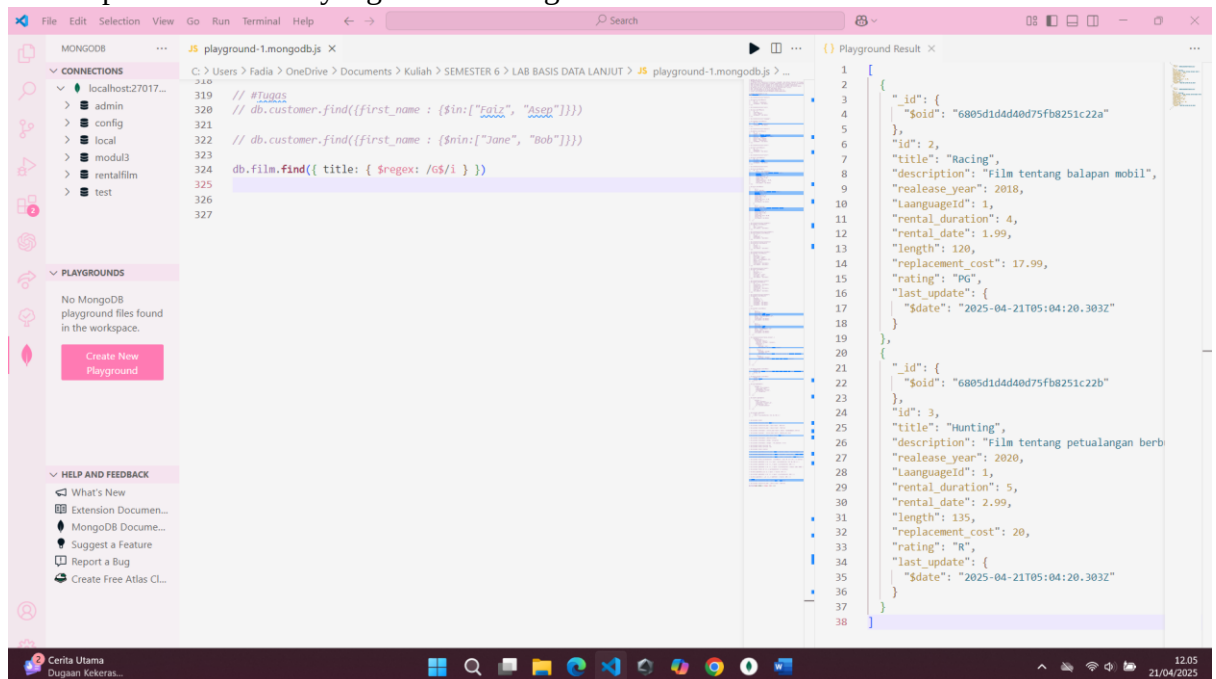
```
286 // db.customer.updateOne( { _id: 1 }, { $pull: { purchaseScores: 160 } } )
287
288 // db.Elem.updateOne( { _id: 1 }, { $addToSet: { results: 160 } } )
289
290 // #Tugas
291 db.customer.find( { first_name : { $in: ["Faiz", "Asep"]} })
292
```
- Results:** The results pane shows two documents from the 'customer' collection. The first document has a first_name of 'Faiz' and a last_name of 'Haidar'. The second document has a first_name of 'Asep' and a last_name of 'Haidar'.

Bottom Screenshot:

- Commands:**

```
272 // db.customer.updateMany( { first_name : "Asep" }, { $set : { last_name : "Ubed" } })
273
274 // db.customer.find( { purchaseScores: { $elemMatch: { $gt: 80, $lt: 85 } } } ).pretty()
275
276 // db.customer.updateOne( { _id: 1 }, { $set: { purchaseScores: [70, 80, 90] } } )
277
278 // db.customer.updateOne( { _id: 1 }, { $push: { purchaseScores: 100 } } )
279
280 // db.customer.updateOne( { _id: 1 }, { $push: { purchaseScores: { $each: [160, 1600] } } } )
281
282 // db.customer.find( { _id: 1 }, { purchaseScores: 1 } ).pretty()
283
284 // db.Elem.updateOne( { _id: 1 }, { $pull: { results: 160 } } )
285
286 // db.customer.updateOne( { _id: 1 }, { $pull: { purchaseScores: 160 } } )
287
288 // db.Elem.updateOne( { _id: 1 }, { $addToSet: { results: 160 } } )
289
290 // #Tugas
291 // db.customer.find( { first_name : { $in: ["Faiz", "Asep"]} })
292
293 db.customer.find( { first_name : { $nin: ["Jane", "Bob"]} })
294
```
- Results:** The results pane shows two documents from the 'customer' collection. The first document has a first_name of 'Alice' and a last_name of 'Johnson'. The second document has a first_name of 'Faiz' and a last_name of 'Haidar'.

2. Buatlah dan Jalankan perintah untuk mencari data dari collection Film untuk menampilkan data Title yang diakhiri dengan huruf G.



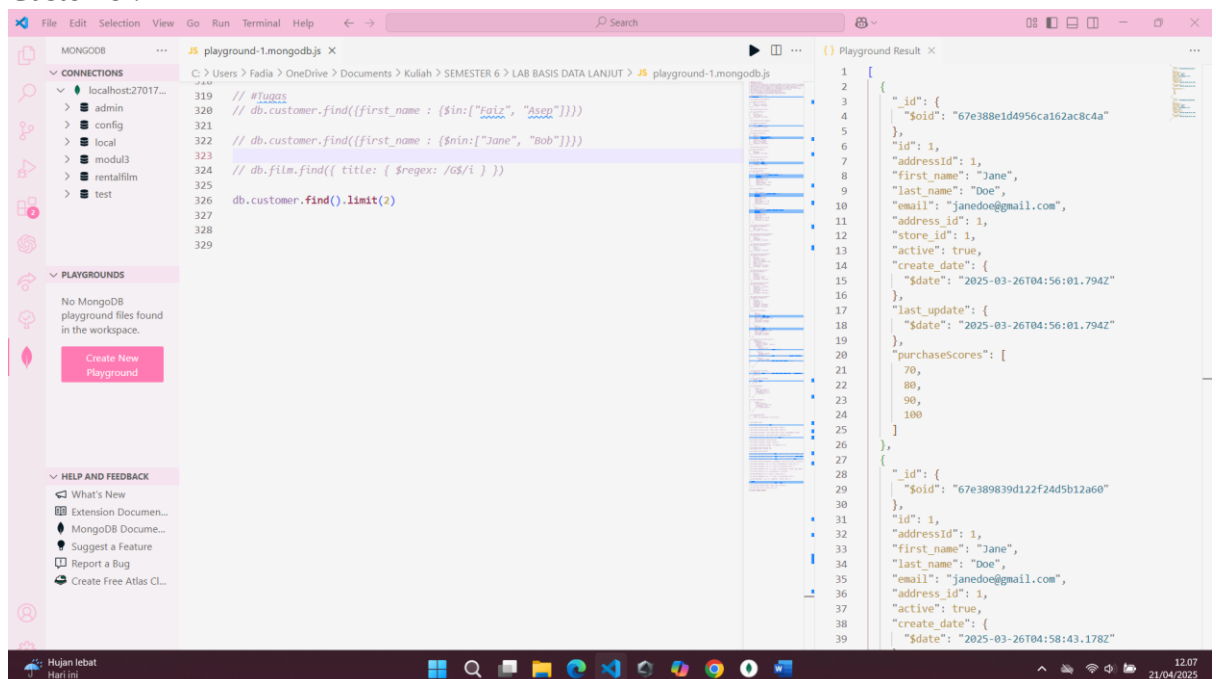
The screenshot shows the MongoDB Playground interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017'. The 'PLAYGROUNDS' panel shows 'No MongoDB playground files found in the workspace.' The main editor shows the following JavaScript code:

```
319 // #Tugas
320 // db.customer.find((first_name : {$in:["Faiz", "Asep"]})))
321
322 // db.customer.find((first_name : {$in:["Jane", "Bob"]})))
323
324 db.film.find( { title: { $regex: /G/i } })
325
326
327
```

The 'PLAYGROUND RESULT' panel shows the following JSON output:

```
1 [
2   {
3     "_id": {
4       "$oid": "6805d1d4d40d75fb8251c22a"
5     },
6     "id": 2,
7     "title": "Racing",
8     "description": "Film tentang balapan mobil",
9     "release_year": 2018,
10    "languageId": 1,
11    "rental_duration": 4,
12    "rental_date": 1.99,
13    "length": 120,
14    "replacement_cost": 17.99,
15    "rating": "PG",
16    "last_update": {
17      "$date": "2025-04-21T05:04:20.303Z"
18    }
19  },
20  {
21    "_id": {
22      "$oid": "6805d1d4d40d75fb8251c22b"
23    },
24    "id": 3,
25    "title": "Hunting",
26    "description": "Film tentang petualangan berb",
27    "release_year": 2020,
28    "languageId": 1,
29    "rental_duration": 5,
30    "rental_date": 2.99,
31    "length": 135,
32    "replacement_cost": 20,
33    "rating": "R",
34    "last_update": {
35      "$date": "2025-04-21T05:04:20.303Z"
36    }
37  }
38 ]
```

3. Buatlah dan jalankan perintah untuk menampilkan 2 buah data dari collection Customer.



The screenshot shows the MongoDB Playground interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017'. The 'PLAYGROUNDS' panel shows 'No MongoDB playground files found in the workspace.' The main editor shows the following JavaScript code:

```
319 // #Tugas
320 // db.customer.find((first_name : {$in:["Faiz", "Asep"]})))
321
322 // db.customer.find((first_name : {$in:["Jane", "Bob"]})))
323
324 // db.film.find( { title: { $regex: /G/i } })
325
326 db.customer.find().limit(2)
327
328
329
```

The 'PLAYGROUND RESULT' panel shows the following JSON output:

```
1 [
2   {
3     "_id": {
4       "$oid": "67e388e1d4956ca162ac8c4a"
5     },
6     "id": 1,
7     "addressId": 1,
8     "first_name": "Jane",
9     "last_name": "Doe",
10    "email": "janedoe@gmail.com",
11    "address_id": 1,
12    "store_id": 1,
13    "active": true,
14    "create_date": {
15      "$date": "2025-03-26T04:56:01.794Z"
16    },
17    "last_update": {
18      "$date": "2025-03-26T04:56:01.794Z"
19    },
20    "purchaseScores": [
21      70,
22      80,
23      90,
24      100
25    ]
26  },
27  {
28    "_id": {
29      "$oid": "67e389839d122f24d5b12a60"
30    },
31    "id": 1,
32    "addressId": 1,
33    "first_name": "Jane",
34    "last_name": "Doe",
35    "email": "janedoe@gmail.com",
36    "address_id": 1,
37    "active": true,
38    "create_date": {
39      "$date": "2025-03-26T04:58:43.178Z"
40    }
41  }
42 ]
```

4. Buatlah dan jalankan perintah untuk menampilkan semua data di film berdasarkan urutan alfabet.

The screenshot shows the MongoDB Playground interface. On the left, the 'CONNECTIONS' panel shows a connection to 'localhost:27017'. The 'PLAYGROUNDS' panel shows a message 'No MongoDB playground files found in the workspace.' and a 'Create New Playground' button. The 'HELP AND FEEDBACK' panel is also visible. The main editor shows a JavaScript playground with the following code:

```
319 // #tugas
320 // db.customer.find({first_name : {$in:["Faiz", "Asep"]}})
321
322 // db.customer.find({first_name : {$nin:["Jane", "Bob"]}})
323
324 // db.film.find({ title: { $regex: /G$/i } })
325
326 // db.customer.find().limit(2)
327
328 db.film.find().sort({ title: 1 })
329
330
```

The 'PLAYGROUND RESULT' panel on the right shows the output of the query, displaying two film documents sorted by title in ascending order:

```
1 [
2   {
3     "_id": {
4       "$oid": "6805d1d4d40d75fb8251c22b"
5     },
6     "id": 3,
7     "title": "Hunting",
8     "description": "Film tentang petualangan berb
9     "release_year": 2020,
10    "languageId": 1,
11    "rental_duration": 5,
12    "rental_date": 2.99,
13    "length": 135,
14    "replacement_cost": 20,
15    "rating": "R",
16    "last_update": {
17      "$date": "2025-04-21T05:04:20.303Z"
18    }
19  },
20  {
21    "_id": {
22      "$oid": "6805d1d4d40d75fb8251c22a"
23    },
24    "id": 2,
25    "title": "Racing",
26    "description": "Film tentang balapan mobil",
27    "release_year": 2018,
28    "languageId": 1,
29    "rental_duration": 4,
30    "rental_date": 1.99,
31    "length": 120,
32    "replacement_cost": 17.99,
33    "rating": "PG",
34    "last_update": {
35      "$date": "2025-04-21T05:04:20.303Z"
36    }
37  },
38  {
39    "_id": {
```

5. Buatlah collection baru dengan nama ArrTest masukan beberapa data yang memiliki setidaknya 1 array kemudian buat dan jalankan perintah Push, Pull dan addToSet secara bergiliran.

The screenshot shows the MongoDB Compass interface. The 'CONNECTIONS' panel on the left shows a connection to 'localhost:27017'. The 'COMPASS' panel on the right shows the 'Documents' tab for the 'rentalfilm' database. A new collection named 'ArrTest' has been created. The 'Documents' tab shows a single document with the following structure:

```
1 {
2   "acknowledged": true,
3   "insertedId": 1
4 }
```

\$Push

The screenshot displays the MongoDB Compass interface. On the left, a JavaScript playground script is shown with the following code:

```
sh > SEMESTER 6 > LAB BASIS DATA LANJUT > playground-1.mongodb.js > ...
322 // db.customer.find({first_name : {$nin:["Jan"]}})
323
324 // db.film.find({ title: { $regex: /GS/i } })
325
326 // db.customer.find().limit(2)
327
328 // db.film.find().sort({ title: 1 })
329
330 // db.createCollection("ArrTest")
331 // db.ArrTest.insertOne({
332 //   _id: 1,
333 //   values: [10, 20, 30]
334 // })
335
336 db.ArrTest.updateOne(
337   { _id: 1 },
338   { $push: { values: 40 } }
339 )
340
341
```

The right pane shows the document in the database:

```
{
  "_id": 1,
  "values": [
    10,
    20,
    30,
    40
  ]
}
```

\$Pull

The screenshot displays the MongoDB Compass interface. On the left, a JavaScript playground script is shown with the following code:

```
Kuliah > SEMESTER 6 > LAB BASIS DATA LANJUT > playground-1.mongodb.js > ...
322 // db.customer.find({first_name : {$nin:["Jan"]}})
323
324 // db.film.find({ title: { $regex: /GS/i } })
325
326 // db.customer.find().limit(2)
327
328 // db.film.find().sort({ title: 1 })
329
330 // db.createCollection("ArrTest")
331 // db.ArrTest.insertOne({
332 //   _id: 1,
333 //   values: [10, 20, 30]
334 // })
335
336 // db.ArrTest.updateOne(
337 //   { _id: 1 },
338 //   { $push: { values: 40 } }
339 // )
340
341 db.ArrTest.updateOne(
342   { _id: 1 },
343   { $pull: { values: 20 } }
344 )
345
346
```

The right pane shows the document in the database:

```
{
  "_id": 1,
  "values": [
    10,
    30,
    40
  ]
}
```

\$addToSet

The screenshot displays the MongoDB Compass interface and a Playground window. The Playground window shows a series of MongoDB commands being executed. The final command is `db.ArrTest.updateOne({ _id: 1 }, { $addToSet: { values: 50 } })`. The Playground Result shows the operation was successful, with `"acknowledged": true`, `"insertedid": null`, `"matchedCount": 1`, `"modifiedCount": 1`, and `"upsertedCount": 0`.

The MongoDB Compass interface shows the `rentalfilm` database and the `ArrTest` collection. The `Documents` tab is selected, showing a single document with the following structure:

```
{
  "_id": 1,
  "values": [
    10,
    20,
    30,
    40,
    50
  ]
}
```

The `values` array now contains the values 10, 20, 30, 40, and 50, demonstrating the effect of the `$addToSet` operator.